

K-Means & K-Medoids

K-Means Clustering Algorithm

- kMeans algorithm is an unsupervised learning algorithm
- Given a data set of items, with certain features, and values for these features, the algorithm will categorize the items into k groups or clusters of similarity.
- To calculate the similarity, we can use the Euclidean distance, Manhattan distance, Hamming distance, Cosine distance as measurement.

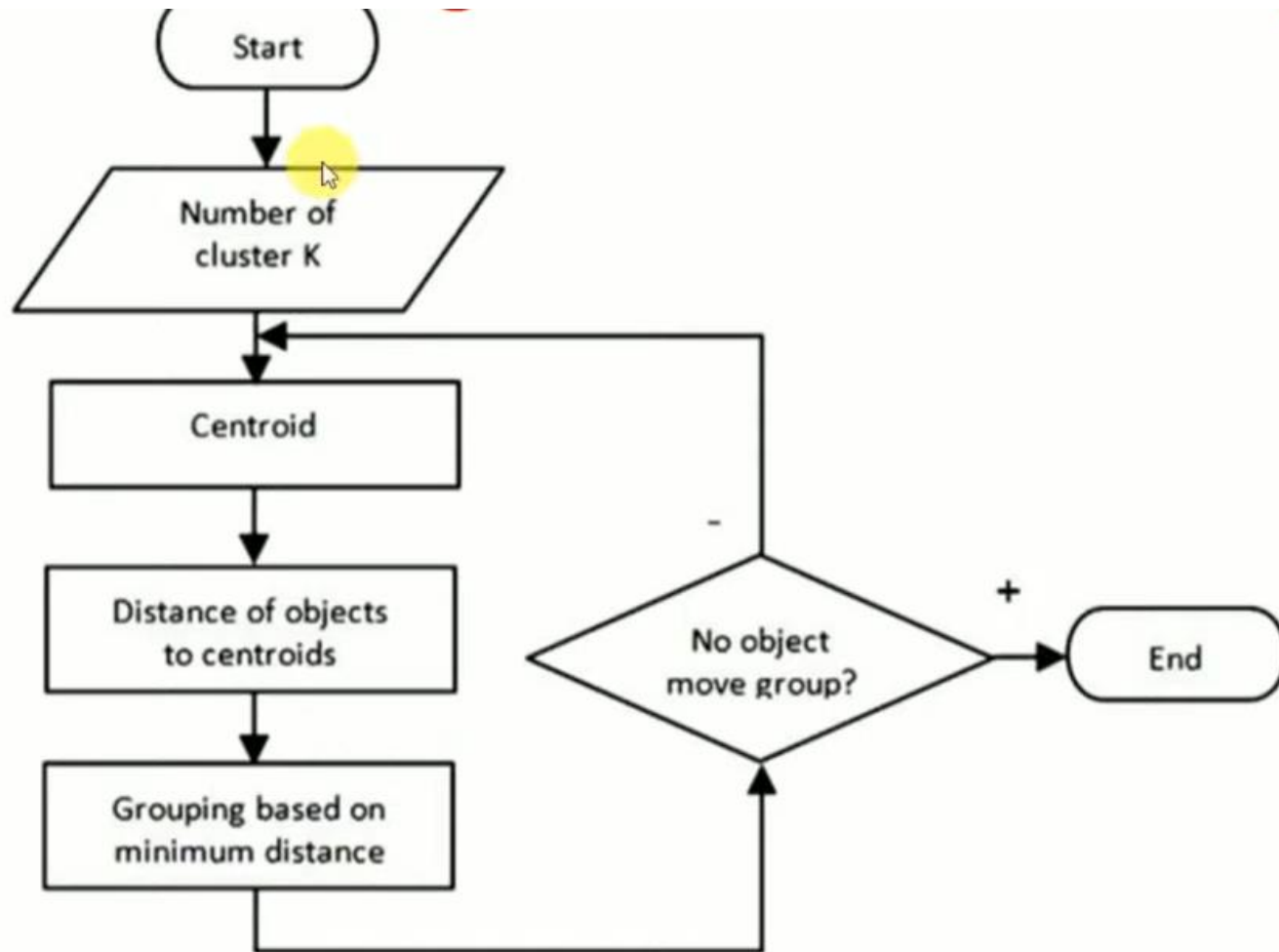
K-Means Algorithm

Here is the pseudocode for implementing a K-means algorithm.

Input: Algorithm K-Means (K number of clusters, D list of data points)

1. Choose K number of random data points as initial centroids (cluster centers).
2. Repeat till cluster centers stabilize:
 - a. Allocate each point in D to the nearest of Kth centroids.
 - b. Compute centroid for the cluster using all points in the cluster.

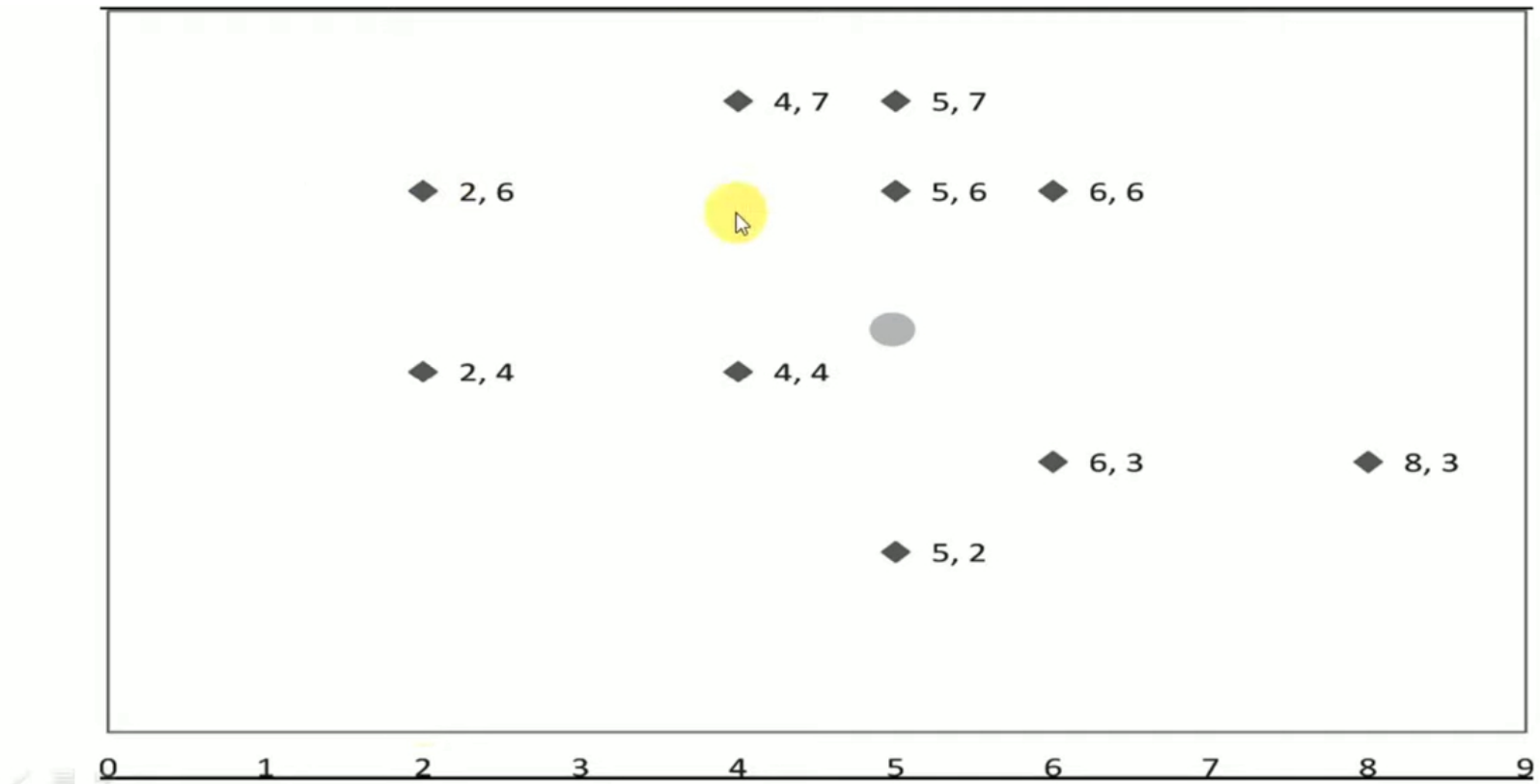
K-Means Algorithm



Solved Example

X	Y
2	4
2	6
5	6
4	7
8	3
6	6
5	2
5	7
6	3
4	4

Solved Example



Solved Example

Iteration - 1

C1 - Seed Point1 – (1, 5)

C2 - Seed Point2 – (4, 1)

C3 - Seed Point3 – (8, 4)

$$D = \sqrt{((x_2 - x_1)^2 + (y_2 - y_1)^2)}$$

C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (4.5, 3)

C3 – Centroid – (6, 5)

X	Y	Distance to			Cluster Number
		(1, 5)	(4, 1)	(8, 4)	
2	4	1.41	3.61	6.00	C1
2	6	1.41	5.39	6.32	C1
5	6	4.12	5.10	3.61	C3
4	7	3.61	6.00	5.00	C1
8	3	7.28	4.47	1.00	C3
6	6	5.10	5.39	2.83	C3
5	2	5.00	1.41	3.61	C2
5	7	4.47	6.08	4.24	C3
6	3	5.39	2.83	2.24	C3
4	4	3.16	3.00	4.00	C2

Solved Example

Iteration - 2

C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (4.5, 3)

C3 – Centroid – (6, 5)

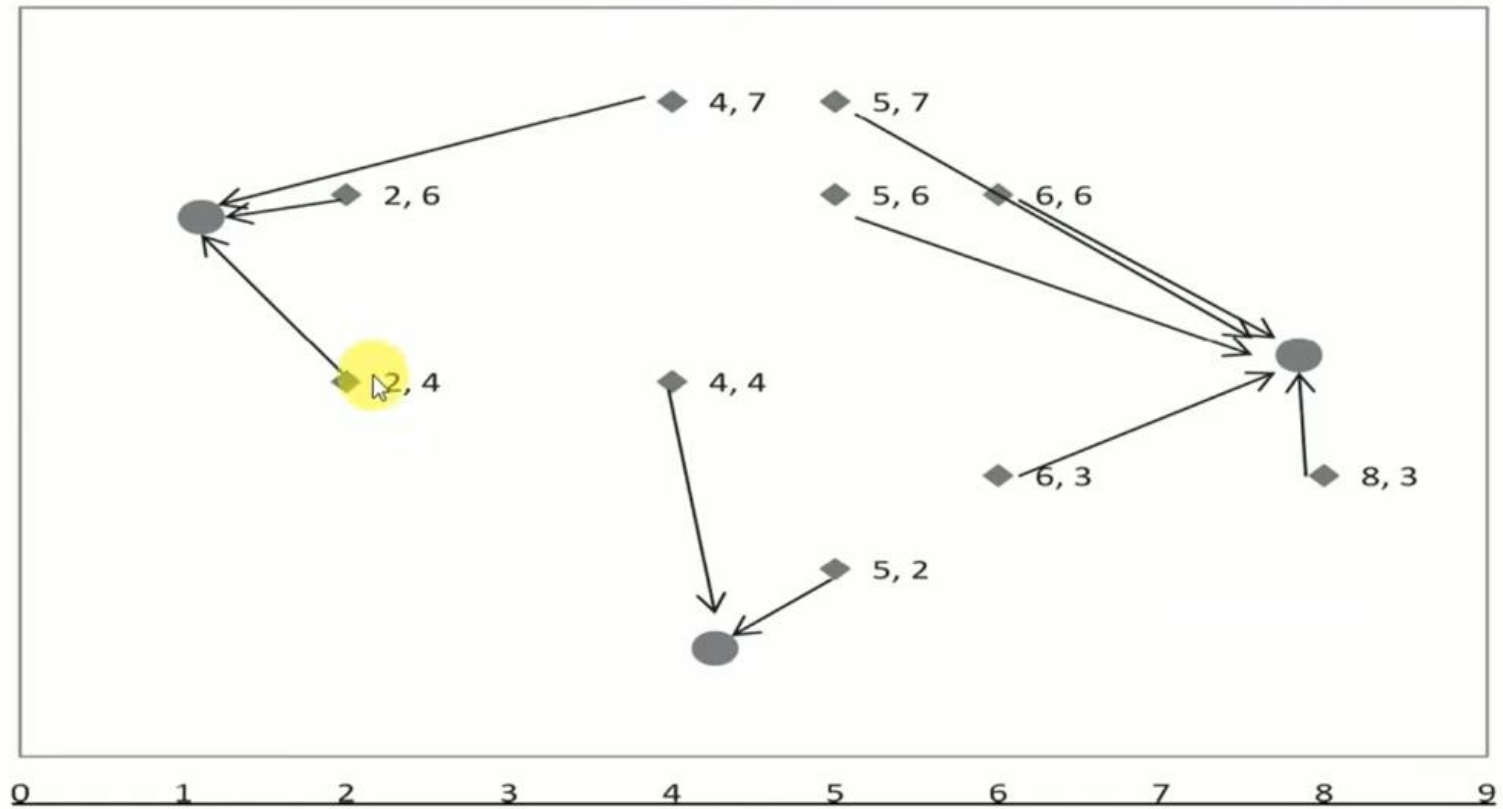
C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (5, 3)

C3 – Centroid – (6, 5.5)

X	Y	Distance to			Cluster Number
		(2.66, 5.66)	(4.5, 3)	(6, 5)	
2	4	1.79	2.69	4.12	C1
2	6	0.74	3.91	4.12	C1
5	6	2.36	3.04	1.41	C3
4	7	1.90	4.03	2.83	C1
8	3	5.97	3.5	2.83	C3
6	6	3.36	3.35	1	C3
5	2	4.34	1.12	3.16	C2
5	7	2.70	4.03	2.24	C3
6	3	4.27	1.5	2	C2
4	4	2.13	1.12	2.24	C2

Solved Example



Solved Example

Iteration - 3

C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (5, 3)

C3 – Centroid – (6, 5.5)

C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (5.75, 3)

C3 – Centroid – (5.33, 6.33)

		Distance to			Cluster
X	Y	(2.66, 5.66)	(5, 3)	(6, 5.5)	Number
2	4	1.79	3.16	4.27	C1
2	6	0.74	4.24	4.03	C1
5	6	2.36	3.00	1.12	C3
4	7	1.90	4.12	2.50	C1
8	3	5.97	3.00	3.20	C2
6	6	3.36	3.16	0.50	C3
5	2	4.34	1.00	3.64	C2
5	7	2.70	4.00	1.80	C3
6	3	4.27	1.00	2.50	C2
4	4	2.13	1.41	2.50	C2

Solved Example

Iteration - 4

C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (5.75, 3)

C3 – Centroid – (5.33, 6.33)

C1 – Centroid – (2, 5)

C2 – Centroid – (5.75, 3)

C3 – Centroid – (5, 6.5)

		Distance to			Cluster
X	Y	(2.66, 5.66)	(5.75, 3)	(5.33, 6.33)	Number
2	4	1.79	3.88	4.06	C1
2	6	0.74	4.80	3.35	C1
5	6	2.36	3.09	0.47	C3
4	7	1.90	4.37	1.49	C3
8	3	5.97	2.25	4.27	C2
6	6	3.36	3.01	0.75	C3
5	2	4.34	1.25	4.34	C2
5	7	2.70	4.07	0.75	C3
6	3	4.27	0.25	3.40	C2
4	4	2.13	2.02	2.68	C2

Solved Example

Iteration - 5

C1 – Centroid – (2, 5)

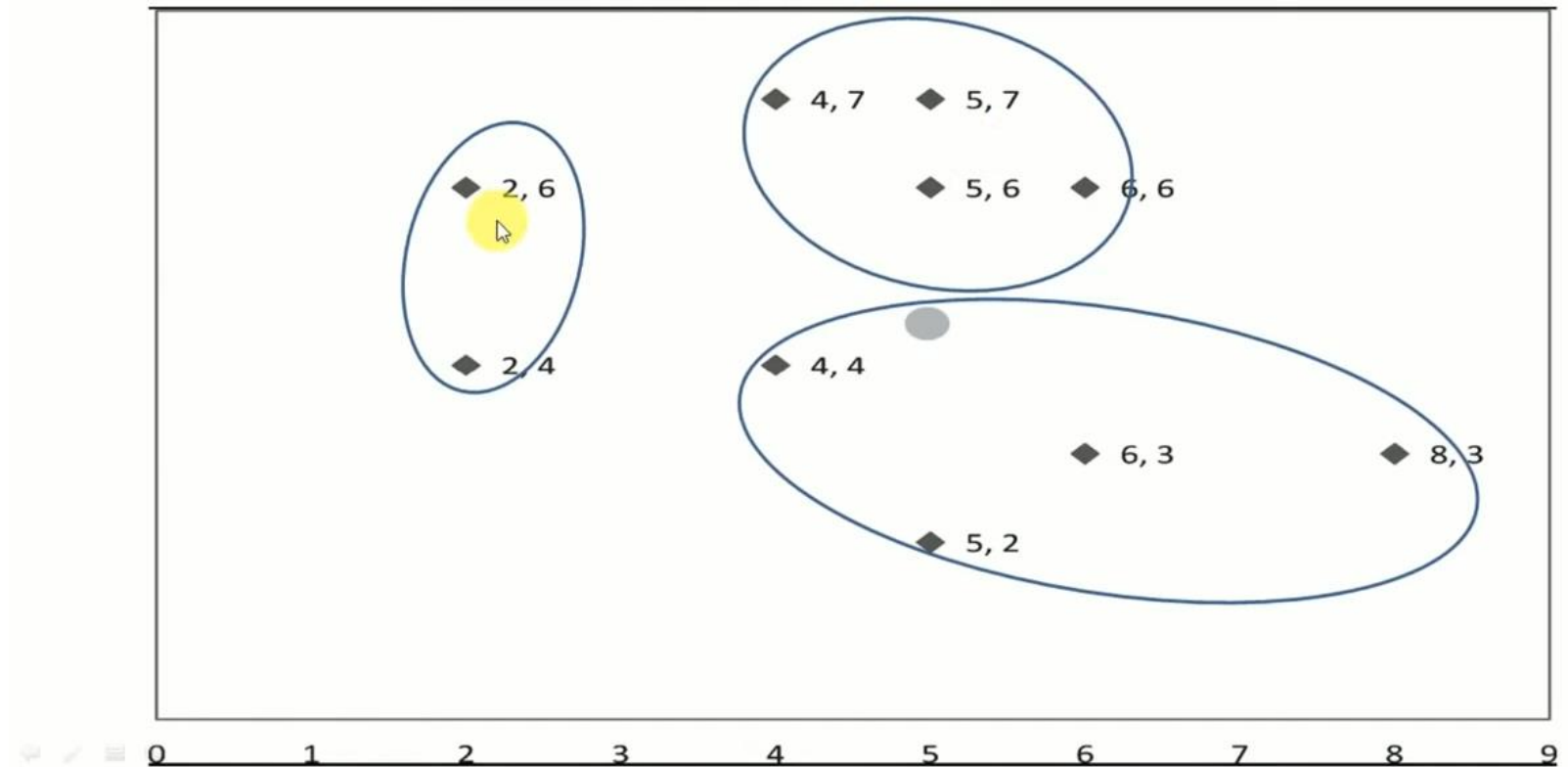
C2 – Centroid – (5.75, 3)

C3 – Centroid – (5, 6.5)

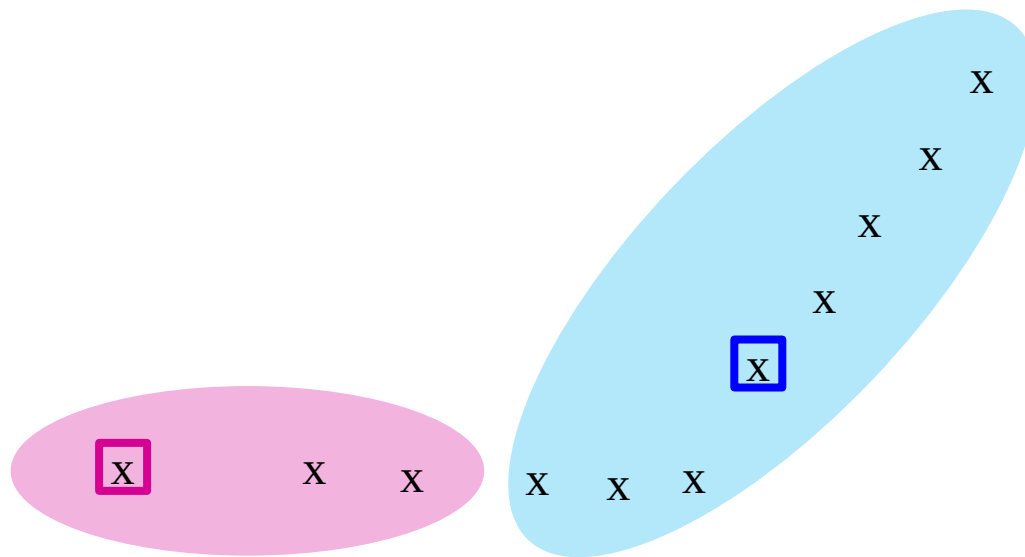
No movement of data Points
Hence these are the final
positions

X	Y	Distance to			Cluster Number
		(2, 5)	(5.75, 3)	(5, 6.5)	
2	4	1.00	3.88	3.91	C1
2	6	1.00	4.80	3.04	C1
5	6	3.16	3.09	0.50	C3
4	7	2.83	4.37	1.12	C3
8	3	6.32	2.25	4.61	C2
6	6	4.12	3.01	1.12	C3
5	2	4.24	1.25	4.50	C2
5	7	3.61	4.07	0.50	C3
6	3	4.47	0.25	3.64	C2
4	4	2.24	2.02	2.69	C2

Solved Example



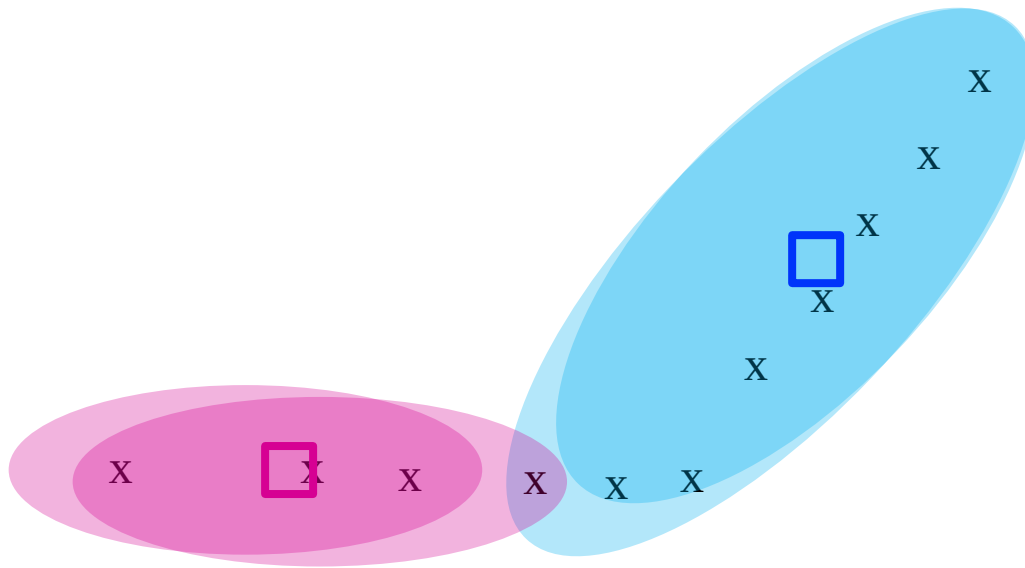
Example: Assigning Clusters



x ... data point
□ ... centroid

Clusters after round 1

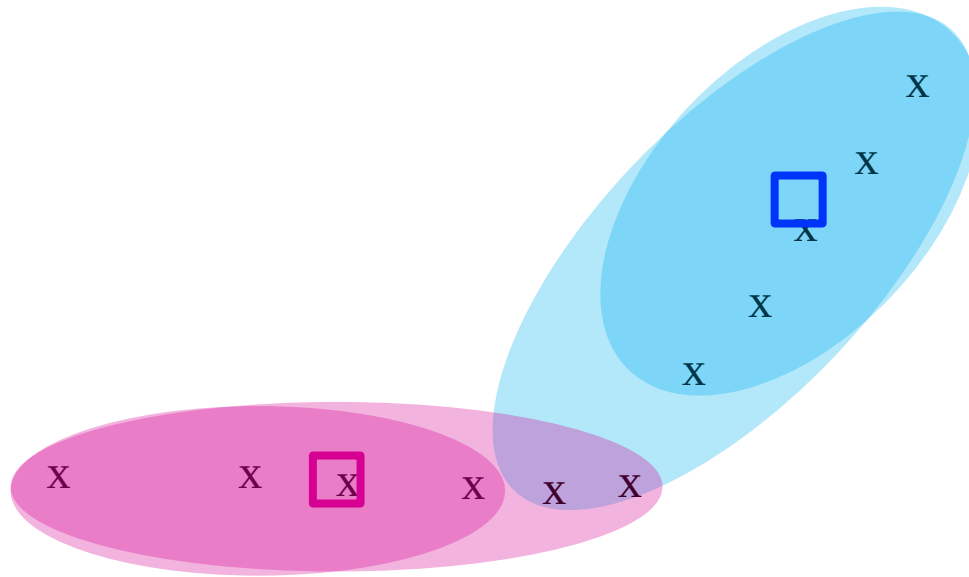
Example: Assigning Clusters



x ... data point
□ ... centroid

Clusters after round 2

Example: Assigning Clusters



x ... data point
□ ... centroid

Clusters at the end

Advantages & Disadvantages of K-Means

Advantages of K-Means Algorithm

1. K-means algorithm is simple, easy to understand, and easy to implement.
2. It is also efficient, in which the time taken to cluster K-means rises linearly with the number of data points.
3. No other clustering algorithm performs better than K-means.

Disadvantages of K-Means Algorithm

1. The user needs to specify an initial value of K.
2. The process of finding the clusters may not converge.
3. It is not suitable for discovering clusters that are not hyper ellipsoids or hyper spheres).
4. It is affected by noise (Outliers)

K-Medoids

Partitioning Around Medoids or the K-Medoids

- is a partitional clustering algorithm which is slightly modified from the K-means algorithm.
- They both attempt to minimize the squared-error but the K-medoids algorithm is more robust to noise than K-means algorithm.
- In **K-means** algorithm, they choose **means** as the centroids but in the **K-medoids**, **data points** are chosen to be the medoids(median).
- A **medoid** can be defined as that object of a cluster, whose average dissimilarity to all the objects in the cluster is minimal.

K-Medoid Algorithm

1. Randomly select k points from the data(k is the number of clusters to be formed). These k points would act as our initial medoids.
2. The distances between the medoid points and the non-medoid points are calculated, and each point is assigned to the cluster of its nearest medoid.
3. Calculate the cost as the total sum of the distances(also called dissimilarities) of the data points from the assigned medoid.
4. Swap one medoid point with a non-medoid point(from the same cluster as the medoid point) and recalculate the cost.
5. If the calculated cost with the new medoid point is more than the previous cost, we undo the swap, and the algorithm converges else; we repeat step 4

Distance Metrics

- Manhattan Distance between two points (x_1, y_1) and (x_2, y_2) is given as
- $Mdist = |x_2 - x_1| + |y_2 - y_1|$

Example cluster using K-Medoids

X	Y
7	6
2	6
3	8
8	5
7	4
4	7
6	2
7	3
6	4
3	4

Solved example

0	7	6
1	2	6
2	3	8
3	8	5
4	7	4
5	4	7
6	6	2
7	7	3
8	6	4
9	3	4

- **Step #1:** $k = 2$
- Let the randomly selected 2 medoids be C1 -(3, 4) and C2 -(7, 4).
- **Step #2:** Calculating cost.

the dissimilarity of each non-medoid point with the medoids is calculated and tabulated:

Solved Example

X	Y	D1 (3,4)	D2(7,4)
7	6	6	2
2	6	3	7
3	8	4	8
8	5	6	2
7	4	4	0
4	7	4	6
6	2	5	3
7	3	5	1
6	4	3	1
3	4	0	4

C1 -(3, 4) and C2 -(7, 4).

Each point is assigned to the cluster of that medoid whose dissimilarity is less.

The points 1, 2, 5 go to cluster C1 and 0, 3, 6, 7, 8 go to cluster C2.

The cost $C = (3 + 4 + 4) + (3 + 1 + 1 + 2 + 2)$

C = 20

Solved Example

Step #3: Now randomly select one non-medoid point and recalculate the cost. Let the randomly selected point be (7, 3), The dissimilarity of each non-medoid point with the medoids – C1 (3, 4) and C2 (7, 3) is calculated and

tabulated.

X	Y	D1 (3,4)	D2(7,4)	D3(7,3)
7	6	6	2	3
2	6	3	7	8
3	8	4	8	9
8	5	6	2	3
7	4	4	0	1
4	7	4	6	7
6	2	5	3	2
7	3	5	1	0
6	4	3	1	2
3	4	0	4	5

Solved Example

- Each point is assigned to that cluster whose dissimilarity is less. So, the points 1, 2, 5 go to cluster C1 and 0, 3, 6, 7, 8 go to cluster C2.
The cost $C = (3 + 4 + 4) + (2 + 2 + 1 + 3 + 3)$
 $C = 22$
Swap Cost = Present Cost – Previous Cost
 $= 22 - 20 = 2 > 0$
- As the swap cost is not less than zero, we undo the swap. Hence (3, 4) and (7, 4) are the final medoids. The clustering would be in the following way

Summary of solved example

Select initial medoids as (3,4) & (7,4)

- $C1 = \{(3, 4), (2, 6), (3, 8), (4, 7)\}$
- $C2 = \{(7, 4), (7, 6), (8, 5), (7, 4), (6, 2), (7, 3), (6, 4)\}$
- $TotalCost = (3 + 4 + 4) + (2 + 2 + 3 + 1 + 1) = 20$

After selecting medoid as (7,3)

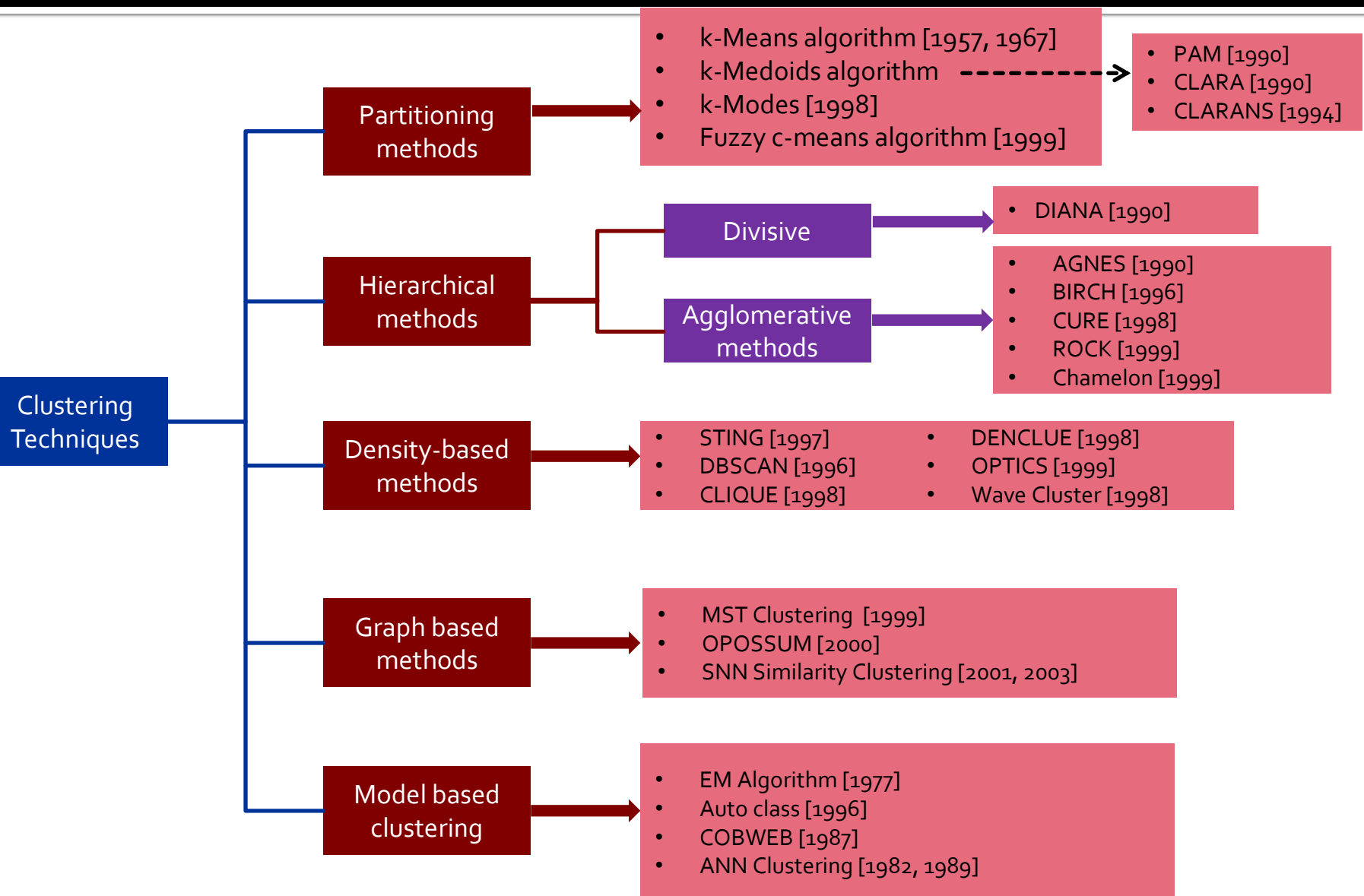
- $C1 = \{(3, 4), (2, 6), (3, 8), (4, 7)\}$
- $C2 = \{(7, 3), (7, 6), (8, 5), (7, 4), (6, 2), (7, 4), (6, 4)\}$
- $TotalCost = (3 + 3 + 4) + (3 + 3 + 1 + 2 + 2) = 22 > 20$
- Hence no change

Advantages & Disadvantages

- **Advantages:**
- It is simple to understand and easy to implement.
- K-Medoid Algorithm is fast and converges in a fixed number of steps.
- PAM is less sensitive to outliers than other partitioning algorithms.
- **Disadvantages:**
- The main disadvantage of K-Medoid algorithms is that it is not suitable for clustering non-spherical (arbitrary shaped) groups of objects. This is because it relies on minimizing the distances between the non-medoid objects and the medoid (the cluster center) – briefly, it uses compactness as clustering criteria instead of connectivity.
- It may obtain different results for different runs on the same dataset because the first k medoids are chosen randomly.

Hierarchical Clustering

Methods of Clustering



Overview: Methods of Clustering

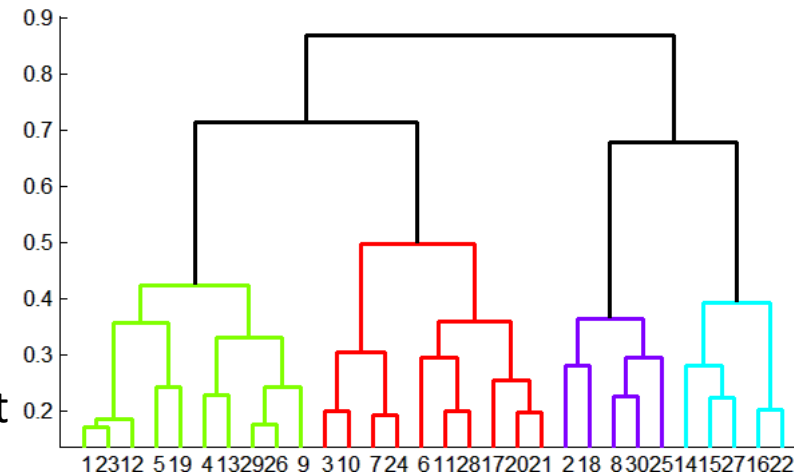
■ Hierarchical:

■ **Agglomerative** (bottom up):

- Initially, each point is a cluster
- Repeatedly combine the two “nearest” clusters into one

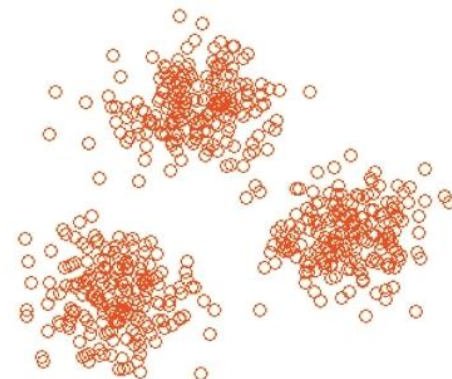
■ **Divisive** (top down):

- Start with one cluster and recursively split it



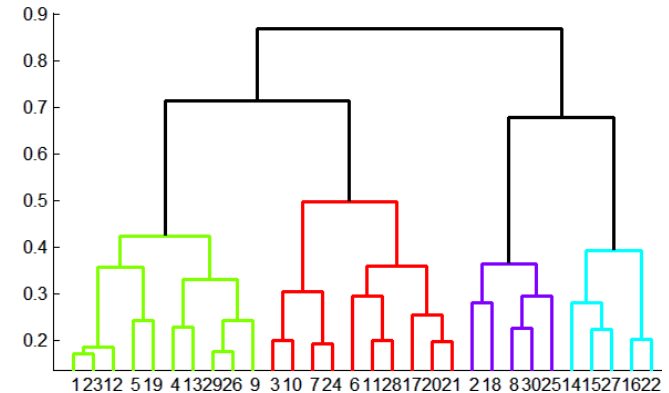
■ **Point assignment:**

- Maintain a set of clusters
- Points belong to “nearest” cluster



Hierarchical Clustering

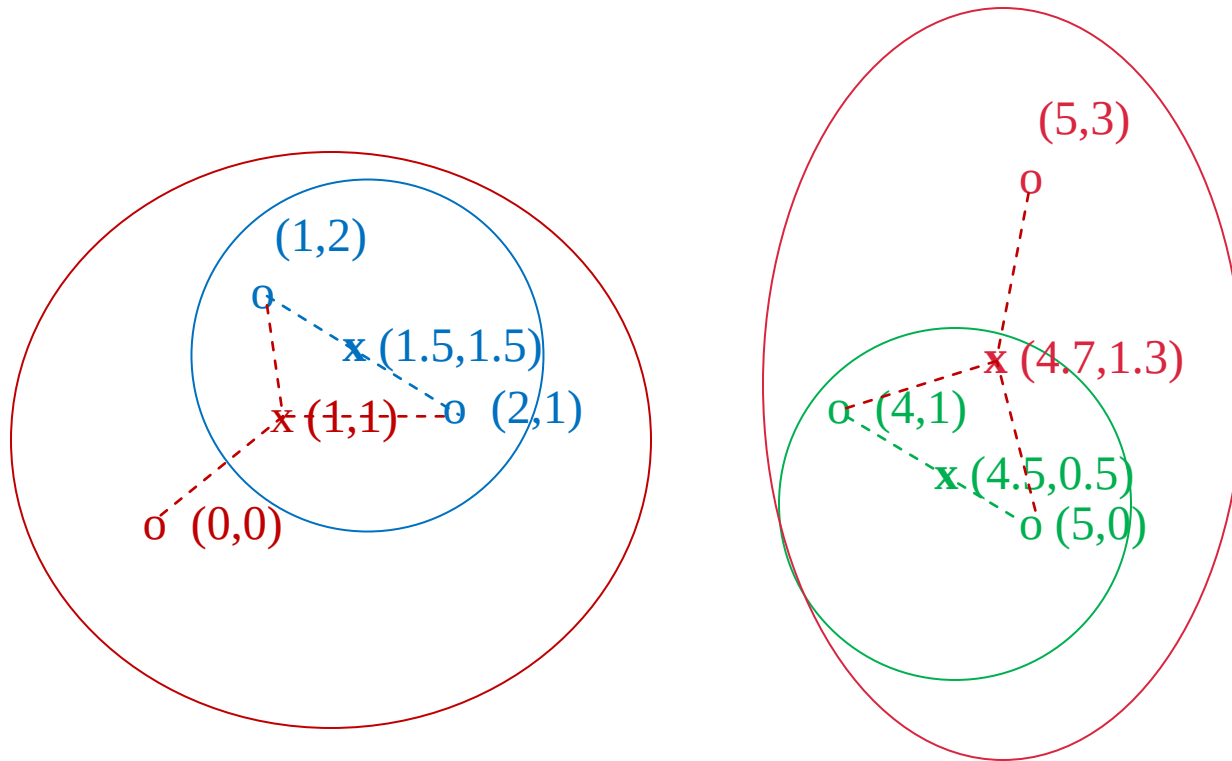
- **Key operation:**
Repeatedly combine
two nearest clusters
- **Three important questions:**
 - 1) How do you represent a cluster of more than one point?
 - 2) How do you determine the “nearness” of clusters?
 - 3) When to stop combining clusters?



Hierarchical Clustering

- **Key operation:** Repeatedly combine two nearest clusters
- **(1) How to represent a cluster of many points?**
 - **Key problem:** As you merge clusters, how do you represent the “location” of each cluster, to tell which pair of clusters is closest?
- **Euclidean case:** each cluster has a **centroid** = average of its (data)points
- **(2) How to determine “nearness” of clusters?**
 - Measure cluster distances by distances of centroids

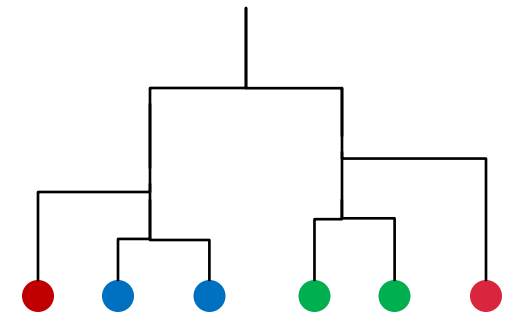
Example: Hierarchical clustering



Data:

σ ... data point

x ... centroid



Dendrogram

And in the Non-Euclidean Case?

What about the Non-Euclidean case?

- The only “locations” we can talk about are the points themselves
 - i.e., there is no “average” of two points
- **Approach 1:**
 - (1) How to represent a cluster of many points?
clustroid = (data)point “closest” to other points
 - (2) How do you determine the “nearness” of clusters? Treat clustroid as if it were centroid, when computing inter-cluster distances

“Closest” Point?

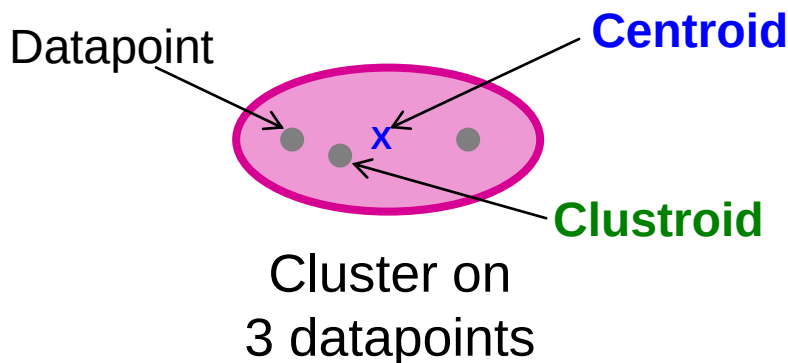
■ (1) How to represent a cluster of many points?

clustroid = point “closest” to other points

■ Possible meanings of “closest”:

- Smallest maximum distance to other points
- Smallest average distance to other points
- Smallest sum of squares of distances to other points
 - For distance metric d clustroid c of cluster C is:

$$\min_c \sum_{x \in C} d(x, c)^2$$



Centroid is the avg. of all (data)points in the cluster. This means centroid is an “artificial” point.

Clustroid is an **existing** (data)point that is “closest” to all other points in the cluster.

Defining “Nearness” of Clusters

- (2) How do you determine the “nearness” of clusters?
 - Approach 2:
Intercluster distance = minimum of the distances between any two points, one from each cluster
 - Approach 3:
Pick a notion of “cohesion” of clusters, *e.g.*, maximum distance from the clustroid
 - Merge clusters whose *union* is most cohesive

Cohesion

- **Approach 3.1:** Use the **diameter** of the merged cluster = maximum distance between points in the cluster
- **Approach 3.2:** Use the **average distance** between points in the cluster
- **Approach 3.3:** Use a **density-based approach**
 - Take the diameter or avg. distance, e.g., and divide by the number of points in the cluster